

A Fast Algorithm for Generating All Triangulations of Biconnected Plane Graphs

Tawkir Aziz Rahman

Department of Computer Science and Engineering
Bangladesh University of Engineering and Technology (BUET)
Dhaka-1000, Bangladesh

October 27, 2025

Abstract

We present a novel algorithm for generating all triangulations of biconnected plane graphs with constant time complexity per triangulation. Previous algorithms for triconnected plane graphs achieve $O(1)$ time per triangulation, but extending these methods to biconnected graphs introduces the fundamental challenge of multi-edge creation during chord flipping operations. Our algorithm addresses this challenge through two key innovations: (1) a **safe root vertex selection** strategy that ensures initial triangulations avoid conflicting chords, and (2) a **local genealogical tree** structure that processes each face independently while maintaining global chord consistency. We prove that our algorithm generates all valid triangulations without duplications, has amortized $O(1)$ time complexity per triangulation, and uses $O(n)$ space, where n is the number of vertices. This work extends the theoretical foundation established by Parvez, Rahman, and Nakano [8] for triconnected graphs to the more general class of biconnected plane graphs, with applications in computational geometry, VLSI floorplanning, and graph drawing.

1 Introduction

Triangulation of planar graphs is a fundamental problem in computational geometry with diverse applications in VLSI floorplanning [5], graph drawing [7], and mesh generation [3]. A *triangulation* of a plane graph is a maximal planar subdivision where every face is bounded by exactly three edges. The problem of generating *all possible triangulations* of a given plane graph presents significant algorithmic challenges due to the exponential number of triangulations and the need to avoid duplications efficiently.

1.1 Background and Motivation

Classical enumeration algorithms either generate objects with duplications and filter them afterward, or use orderly generation [9] to ensure canonical representation. These approaches require substantial memory to store previously generated objects or complex isomorphism checking. The reverse search paradigm [1] addresses memory concerns by

implicitly defining a spanning tree over the solution space, but typically requires $O(n^2)$ time per solution for triangulation problems.

For the specific case of *triconnected* plane graphs, Parvez, Rahman, and Nakano [8] developed an elegant algorithm that generates all triangulations in $O(1)$ time per triangulation by establishing a tree structure among triangulations and using edge flipping operations. Their algorithm decomposes the triconnected graph into faces (cycles), triangulates each face independently, and combines these face triangulations to produce complete graph triangulations.

However, extending this approach to *biconnected* plane graphs—which lack the strong connectivity guarantees of triconnected graphs—introduces a critical obstacle: the **multi-edge problem**. When faces share more than just boundary edges, chord flipping operations can inadvertently create multiple edges between the same pair of vertices, violating the simple graph property. This makes the triconnected algorithm inapplicable to general biconnected graphs.

1.2 Contributions

In this paper, we present the first algorithm for generating all triangulations of biconnected plane graphs with constant amortized time complexity. Our main contributions are:

1. **Safe Root Selection Strategy:** We introduce the concept of a *safe root vertex* for each face—a vertex from which all initial chords can be added without creating multi-edges. We prove that every face in a biconnected plane graph contains at least two safe root vertices, ensuring our algorithm always has a valid starting point.
2. **Local Genealogical Trees:** Instead of a single global tree of triangulations, we construct local genealogical trees for each face. This enables independent face processing while a global chord set prevents multi-edge conflicts across faces.
3. **Theoretical Guarantees:** We rigorously prove that our algorithm:
 - Generates all valid triangulations without duplications (completeness)
 - Always finds a safe root triangulation for any face (existence)
 - Achieves amortized $O(1)$ time per triangulation (efficiency)
 - Uses $O(n)$ space (space efficiency)
4. **Practical Implementation:** We provide a complete implementation demonstrating the algorithm’s effectiveness on real-world graph instances.

1.3 Organization

The remainder of this paper is organized as follows. Section 2 provides necessary definitions and notation. Section 3 discusses related work. Section 4 presents our algorithm in detail. Section 5 proves correctness and completeness. Section 6 analyzes time and space complexity. Section 7 describes implementation details and experimental results. Section 8 concludes with future research directions.

2 Preliminaries

We begin by establishing the graph-theoretic foundations necessary for our algorithm.

Definition 1 (Planar Graph and Plane Graph). A graph $G = (V, E)$ is *planar* if it can be embedded in the plane such that no two edges intersect except at vertices. A *plane graph* is a planar graph with a fixed embedding, dividing the plane into connected regions called *faces*. The unbounded region is the *outer face*; all other faces are *inner faces*.

Definition 2 (Connectivity). The *connectivity* $\kappa(G)$ of a graph G is the minimum number of vertices whose removal disconnects G or reduces it to a single vertex. A graph is *k-connected* if $\kappa(G) \geq k$. In particular:

- A graph is *biconnected* if $\kappa(G) \geq 2$ (no cut vertices).
- A graph is *triconnected* if $\kappa(G) \geq 3$ (no cut vertex pairs).

Definition 3 (Cycle and Chord). A *cycle* C in a plane graph G is a sequence of distinct vertices $v_1, v_2, \dots, v_k$ such that $(v_i, v_{i+1}) \in E$ for $1 \leq i < k$ and $(v_k, v_1) \in E$. We denote $C = \langle v_1, v_2, \dots, v_k \rangle$ with vertices in counterclockwise order.

A *chord* of cycle C is an edge (v_i, v_j) connecting two non-consecutive vertices of C that lies entirely within the face bounded by C .

Definition 4 (Triangulation of a Cycle). A *triangulation* T of a cycle C with n vertices is a maximal set of non-crossing chords that decomposes the interior region of C into triangles. Every triangulation of an n -vertex cycle contains exactly $n - 3$ chords and produces $n - 2$ triangles.

Definition 5 (Triangulation of a Plane Graph). A *triangulation* of a plane graph G is a plane graph T obtained by adding chords to G such that every face of T is bounded by exactly three edges. We say T is a *triangulation of G* .

Definition 6 (Labeled and Unlabeled Triangulations). If the vertices of a cycle C are numbered sequentially from v_1 to v_n , then triangulations of C are called *labeled triangulations*. If vertices are not numbered, triangulations are called *unlabeled triangulations*.

Definition 7 (Visibility). In a triangulation T of a cycle C , we say vertex v_j is *visible from* vertex v_i if there exists a chord (v_i, v_j) in T .

3 Related Work

3.1 Triangulation Enumeration

Early work on triangulation enumeration focused on specific graph classes. Chazelle [2] gave a linear-time algorithm for finding *a* triangulation of a simple polygon, but not for enumerating all triangulations.

Hurtado and Noy [4] studied the graph of triangulations of convex polygons, where vertices represent triangulations and edges represent edge flips. They established theoretical properties but did not provide efficient enumeration algorithms for arbitrary n -vertex polygons without first generating all smaller cases.

Li and Nakano [6] developed an algorithm for generating all biconnected plane triangulations with at most n vertices, but this requires generating all smaller triangulations first—inefficient when only triangulations of exactly n vertices are needed.

3.2 Reverse Search and Tree-Based Enumeration

The reverse search paradigm [1] has been successfully applied to many enumeration problems. Avis presented algorithms for enumerating triangulations using reverse search, but these achieve $O(n^2)$ time per triangulation for cycles [1].

Orderly generation [9] avoids duplicates by outputting only canonical representations, but requires expensive isomorphism testing.

3.3 The Parvez-Rahman-Nakano Algorithm

The breakthrough work of Parvez, Rahman, and Nakano [8] introduced the *tree of triangulations* for triconnected plane graphs, achieving $O(1)$ time per triangulation. Their key insights include:

- **Tree Structure:** Triangulations form a tree where each child is generated from its parent by a single edge flip.
- **Root Triangulation:** The tree is rooted at a triangulation where all chords emanate from a single vertex.
- **Generating Chord:** Each triangulation has a unique *generating chord*—the leftmost chord that can be flipped.
- **Parent-Child Relationship:** Flipping the generating chord produces the parent, ensuring unique relationships and no duplicates.

However, this algorithm is specifically designed for *triconnected* graphs. The triconnectivity guarantee ensures that faces share only boundary edges, preventing multi-edge conflicts. For biconnected graphs without this guarantee, the algorithm fails.

3.4 The Multi-Edge Problem

In biconnected graphs, faces can share internal chords from previous triangulation steps. When triangulating a new face, adding a chord that already exists as a chord in a different face creates a multi-edge, violating the simple graph property. This is the fundamental barrier preventing direct application of the Parvez-Rahman-Nakano algorithm to biconnected graphs.

Our algorithm solves this problem through safe root selection and global chord tracking.

3.5 Comparison: Triconnected vs. Biconnected Approach

The key differences between the Parvez-Rahman-Nakano algorithm for triconnected graphs and our algorithm for biconnected graphs are:

Our algorithm generalizes the triconnected case: when applied to a triconnected graph, every vertex is a safe root (since no chords exist between faces), and the algorithm behaves similarly to the Parvez-Rahman-Nakano algorithm.

Aspect	Triconnected (PRN)	Biconnected (Ours)
Tree Structure	Single global tree of triangulations	Local trees per face + DFS combination
Root Selection	Any vertex can be root	Must find safe root vertex
Multi-edge Problem	Does not occur (guaranteed by triconnectivity)	Explicitly handled via Global-ChordSet
Face Processing	Can process independently	Must process sequentially with backtracking
Chord Conflicts	No conflicts between faces	Dynamic conflict checking required
Complexity	$O(1)$ per triangulation	$O(1)$ amortized per triangulation
Applicability	Only triconnected graphs	All biconnected graphs

Table 1: Comparison of triconnected and biconnected triangulation algorithms

4 The Algorithm

4.1 Algorithm Overview

Our algorithm generates all triangulations of a biconnected plane graph G with k faces $F_1, F_2, \dots, F_k$ using a recursive depth-first search (DFS) strategy. The key idea is to:

1. Process faces sequentially from F_1 to F_k .
2. For each face F_i , construct a *local genealogical tree* of all its valid triangulations.
3. Maintain a *global chord set* to prevent multi-edge conflicts.
4. Recursively combine face triangulations to produce complete graph triangulations.

4.2 Key Innovations

Our algorithm introduces three key innovations to extend the Parvez-Rahman-Nakano framework from triconnected to biconnected graphs:

4.2.1 Innovation 1: Local Genealogical Trees

Instead of constructing a single global genealogical tree for the entire graph (which fails for biconnected graphs due to multi-edge conflicts), we build *local genealogical trees* for each face independently.

- Each face F_i has its own tree $\mathcal{T}(F_i)$ of triangulations
- The root of $\mathcal{T}(F_i)$ is the safe root triangulation
- Parent-child relationships are defined by edge flips within the face
- Local trees are explored via DFS during recursive face processing

This decomposition enables independent triangulation of each face while maintaining global consistency through the chord set.

4.2.2 Innovation 2: Safe Root Vertex Selection

To ensure that the root triangulation of each face does not create multi-edges with previously triangulated faces, we introduce the concept of a *safe root vertex*.

The safe root vertex is chosen such that all initial chords from this vertex to non-neighboring vertices of the face do not conflict with chords already in the global set. This guarantees a valid starting point for the local genealogical tree.

We prove (Theorem 1) that every face, regardless of how many previous faces have been triangulated, contains at least two safe root vertices.

4.2.3 Innovation 3: Global Chord Set with Dynamic Management

The *global chord set* S tracks all chords currently used in the partial triangulation being explored. It serves two purposes:

1. **Conflict Detection:** Before adding any chord to a face triangulation, check if it already exists in S
2. **Safe Root Finding:** Used to identify which vertices can serve as safe roots for the current face

The set is managed dynamically during DFS:

- **Push:** When exploring a triangulation, add its chords to S
- **Recurse:** Process next face with updated S
- **Pop:** After exploring all descendants, remove chords from S (backtrack)

This ensures that S always reflects exactly the chords in the current exploration path.

4.3 Safe Root Vertex Selection

The first critical innovation is identifying a *safe root vertex* for each face.

Definition 8 (Safe Root Vertex). A vertex r of a face F_i is a *safe root vertex* if adding chords from r to all non-neighboring vertices of F_i does not create any edge that already exists in the global chord set (from previous faces $F_1, \dots, F_{i-1}$).

We will prove in Section 5 that every face has at least two safe root vertices.

4.4 Root Triangulation

Once a safe root vertex r is identified, we construct the *root triangulation* of the face by adding chords from r to all non-neighboring vertices:

Definition 9 (Root Triangulation). The *root triangulation* T_r of a face F with safe root r is the triangulation formed by adding all chords (r, v) for every vertex $v \in F$ that is not a neighbor of r on the cycle boundary.

Algorithm 1 Finding a Safe Root Vertex

Require: Face F with vertices $v_1, v_2, \dots, v_n$, global chord set S

Ensure: A safe root vertex r

```
1: for  $i = 1$  to  $n$  do
2:    $\text{safe} \leftarrow \text{true}$ 
3:   for each non-neighbor  $v_j$  of  $v_i$  in  $F$  do
4:     if  $(v_i, v_j) \in S$  then
5:        $\text{safe} \leftarrow \text{false}$ 
6:       break
7:     end if
8:   end for
9:   if  $\text{safe}$  then
10:    return  $v_i$ 
11:   end if
12: end for
```

4.5 Local Genealogical Tree

For each face, we build a *local genealogical tree* of triangulations rooted at the safe root triangulation. The tree structure is defined by parent-child relationships based on edge flipping:

Definition 10 (Generating Chord). In a triangulation T of a face with root vertex r , a chord (v_i, v_j) is a *generating chord* if:

1. Neither v_i nor v_j is the root vertex r , and
2. (v_i, v_j) is the leftmost such chord (smallest index among non-root chords).

Definition 11 (Parent-Child Relationship). Let T be a triangulation with generating chord (v_i, v_j) forming two triangles (v_i, v_j, v_p) and (v_i, v_j, v_q) on opposite sides. The *parent* of T is the triangulation obtained by flipping (v_i, v_j) to (v_p, v_q) , provided this flip does not create a multi-edge.

4.6 Main Algorithm

The algorithm performs a depth-first exploration where:

- At each level i , we process face F_i
- For each valid triangulation T of F_i , we recursively process F_{i+1}
- Backtracking removes chords from S to explore alternative triangulations
- Complete triangulations are output when $i > k$

4.7 Child Generation via Edge Flipping

To generate children in the local genealogical tree, we flip the generating chord:

Algorithm 2 Generate All Triangulations of Biconnected Plane Graph

Require: Biconnected plane graph G with faces $F_1, \dots, F_k$

Ensure: All triangulations of G

```
1: Initialize global chord set  $S \leftarrow \emptyset$ 
2: PROCESSFACE(1,  $\emptyset$ ,  $S$ )
3: procedure PROCESSFACE( $i, T_{\text{current}}, S$ )
4:   if  $i > k$  then
5:     output  $T_{\text{current}}$  ▷ All faces triangulated
6:     return
7:   end if
8:    $r \leftarrow \text{FINDSAFEROOT}(F_i, S)$ 
9:    $T_{\text{root}} \leftarrow \text{ROOTTRIANGULATION}(F_i, r)$ 
10:  EXPLORE( $F_i, T_{\text{root}}, i, T_{\text{current}}, S$ )
11: end procedure
12: procedure EXPLORE( $F, T, i, T_{\text{current}}, S$ )
13:  Add chords of  $T$  to  $S$ 
14:  PROCESSFACE( $i + 1, T_{\text{current}} \cup T, S$ )
15:  Remove chords of  $T$  from  $S$ 
16:  for each child  $T'$  of  $T$  in local tree do
17:    if chords of  $T'$  do not conflict with  $S$  then
18:      EXPLORE( $F, T', i, T_{\text{current}}, S$ )
19:    end if
20:  end for
21: end procedure
```

Algorithm 3 Generate Children of Triangulation

Require: Triangulation T of face F with root r

Ensure: All children of T in the local tree

```
1: Find generating chord  $(v_i, v_j)$  of  $T$ 
2: if no generating chord exists then
3:   return  $\emptyset$  ▷  $T$  is a leaf node
4: end if
5: Find  $v_p, v_q$  forming triangles with  $(v_i, v_j)$ 
6:  $T' \leftarrow T \setminus \{(v_i, v_j)\} \cup \{(v_p, v_q)\}$ 
7: return  $\{T'\}$ 
```

Algorithm 4 Find Safe Root Vertex

```
1: procedure FINDSAFEROOTVERTEX( $F, S$ )
2:   Let  $F = \langle v_1, v_2, \dots, v_n \rangle$ 
3:   for  $i = 1$  to  $n$  do
4:     isSafe  $\leftarrow$  true
5:     for  $j = 1$  to  $n$  do
6:       if  $|i - j| > 1$  and  $|i - j| < n - 1$  then  $\triangleright v_j$  is non-neighbor
7:         if  $(v_i, v_j) \in S$  then
8:           isSafe  $\leftarrow$  false
9:           break
10:        end if
11:      end if
12:    end for
13:    if isSafe then
14:      return  $v_i$ 
15:    end if
16:  end for
17:  return null  $\triangleright$  Should never reach here by Theorem 1
18: end procedure
```

Algorithm 5 Construct Root Triangulation

```
1: procedure ROOTTRIANGULATION( $F, r$ )
2:   Let  $F = \langle v_1, v_2, \dots, v_n \rangle$ 
3:    $T \leftarrow \emptyset$ 
4:   for each vertex  $v_j \in F$  do
5:     if  $v_j \neq r$  and  $v_j$  is not a neighbor of  $r$  on  $F$  then
6:        $T \leftarrow T \cup \{(r, v_j)\}$ 
7:     end if
8:   end for
9:   return  $T$ 
10: end procedure
```

Algorithm 6 Find All Triangulations of a Cycle

```
1: procedure FINDALLTRIANGULATIONSCYCLE( $F, T, i, T_{\text{current}}, S$ )
2:   Add all chords of  $T$  to  $S$ 
3:   if  $i = k$  then  $\triangleright$  All faces processed
4:     output  $T_{\text{current}} \cup T$ 
5:   else
6:     PROCESSFACE( $i + 1, T_{\text{current}} \cup T, S$ )
7:   end if
8:   Remove all chords of  $T$  from  $S$ 
9:   Find all child triangulations of  $T$ 
10:  for each child  $T'$  of  $T$  do
11:    FINDALLTRIANGULATIONSCYCLE( $F, T', i, T_{\text{current}}, S$ )
12:  end for
13: end procedure
```

Algorithm 7 Start Triangulation Process

```
1: procedure STARTTRIANGULATION( $G$ )
2:   Decompose  $G$  into faces  $F_1, F_2, \dots, F_k$ 
3:   Initialize global chord set  $S \leftarrow \emptyset$ 
4:   PROCESSFACE(1,  $\emptyset, S$ )
5: end procedure
```

4.8 Complete Algorithm Pseudocode

For completeness, we provide detailed pseudocode for all major components:

5 Correctness and Completeness

We now prove that our algorithm correctly generates all triangulations without duplicates.

5.1 Existence of Safe Root Vertices

Theorem 1 (Existence of Safe Root). *In any face of a biconnected plane graph during the triangulation process, there exist at least two safe root vertices.*

Proof. Consider a face F with n vertices $v_1, v_2, \dots, v_n$ listed in counterclockwise order. Let S be the global chord set from previously processed faces.

Case 1: If S contains no chords with both endpoints in F , then all n vertices are safe roots.

Case 2: Assume S contains at least one chord with both endpoints in F . Consider a chord $(v_a, v_b) \in S$. Due to planarity, no other chord can cross (v_a, v_b) .

The chord (v_a, v_b) divides the cycle F into two segments:

- $S_1 = \langle v_a, v_{a+1}, \dots, v_b \rangle$
- $S_2 = \langle v_b, v_{b+1}, \dots, v_a \rangle$

Each segment forms a substructure T_m where m is the number of internal vertices (vertices excluding the endpoints).

Base Case: T_1

If S_1 has one internal vertex v_c between v_a and v_b , then both v_a and v_b are neighbors of v_c on the boundary. Any potential chord from v_c would go to one of its neighbors, which is not allowed. Therefore, v_c has no conflicting chords and is a safe root.

Base Case: T_2

If S_1 has two internal vertices v_c and v_d (with v_c between v_a and v_d , and v_d between v_c and v_b), then:

- The only non-neighbor of v_c is v_b
- The only non-neighbor of v_d is v_a

If $(v_c, v_b) \in S$, this creates a T_1 structure containing only v_d , making v_d safe. If $(v_d, v_a) \in S$, this creates a T_1 structure containing only v_c , making v_c safe. If neither chord exists, both v_c and v_d are safe.

Inductive Case: T_m for $m > 2$

We use an iterative reduction strategy. Consider the first internal vertex v_{a+1} :

- Its neighbors on the boundary are v_a and v_{a+2}
- Possible conflicting chords: $(v_{a+1}, v_{a+3}), \dots, (v_{a+1}, v_b)$

Check from v_b downward:

- If $(v_{a+1}, v_b) \in S$, the structure reduces to T_{m-1} with vertices $v_{a+2}, \dots, v_b$
- If $(v_{a+1}, v_{b-k}) \in S$ for some $k > 0$, the structure reduces to T_{m-k-1}
- If no such chord exists, v_{a+1} is a safe root

This process either finds a safe root or reduces m , eventually reaching T_1 or T_2 , both of which contain safe roots.

Finding Safe Vertices in $O(n)$ Time:

The above iterative process can be implemented efficiently:

1. Consider vertex v_{a+1} (first internal vertex)
2. v_{a+1} cannot create chords with neighbors v_a and v_{a+2}
3. Possible chord partners: $\{v_{a+3}, \dots, v_b\}$
4. Check from v_b downward using a hash set ($O(1)$ per check)
5. If a chord (v_{a+1}, v_{b-k}) is found, reduce to subproblem T_{m-k-1}
6. If no chord is found after checking all candidates, v_{a+1} is safe

Each iteration either:

- Finds a safe root vertex (algorithm terminates successfully), or
- Reduces the problem size by at least 1, eventually reaching T_1 or T_2

Since T_1 and T_2 are proven to contain safe vertices, and each reduction step decreases m , the algorithm is guaranteed to find a safe vertex in $O(n)$ time.

Two Complementary T_m Structures:

The face F is divided by chord (v_a, v_b) into two segments:

- Structure 1: $S_1 = \langle v_a, v_{a+1}, \dots, v_b \rangle$ with m_1 internal vertices
- Structure 2: $S_2 = \langle v_b, v_{b+1}, \dots, v_a \rangle$ with m_2 internal vertices

Note that $m_1 + m_2 = n - 2$ (total vertices minus the two endpoints of the constraining chord).

By the analysis above, each structure S_1 and S_2 contains at least one safe vertex. Therefore, every face has at least two safe root vertices—one from each complementary segment.

Since both segments S_1 and S_2 contain safe roots by this argument, every face has at least two safe root vertices. $\square$

Remark 1. The existence of at least two safe root vertices is crucial for the algorithm's flexibility. It ensures that even in highly constrained faces with many pre-existing chords from previous face triangulations, we can always find a valid starting point for the local genealogical tree.

5.2 Completeness via Pruning

Theorem 2 (Completeness). *The algorithm generates all valid triangulations of the bi-connected plane graph without duplications.*

Proof. We must show two properties:

1. Every valid triangulation is generated (completeness)
2. No triangulation is generated twice (no duplicates)

Pruning Preserves Validity:

When the algorithm prunes a branch (because a chord would create a multi-edge), we must verify that all children of that branch would also be invalid.

Consider a triangulation T with a blocking chord (v_a, v_b) that already exists in the global set S . Since T is a complete triangulation of the face, there exist two vertices v_p and v_q forming triangles (v_a, v_b, v_p) and (v_a, v_b, v_q) on opposite sides of the chord.

We now prove that a blocking chord is *never flipped* in any child of the triangulation tree, by analyzing all possible positions of the vertices relative to the root vertex r .

Case 1: v_p is on the left of the root vertex r

Flipping the (v_a, v_b) chord would result in a (v_p, v_q) chord. However, before this flip, the (v_b, v_p) chord was also a blocking chord positioned to the left of (v_a, v_b) (by the ordering of vertices). This means (v_a, v_b) could not be the leftmost blocking chord.

Consequently, after flipping to create the new (v_p, v_q) chord, this new chord cannot be the leftmost blocking chord either, because the (v_p, v_b) chord still exists and remains to the left of the (v_q, v_p) chord.

Therefore, flipping the (v_a, v_b) blocking chord is not permitted by the algorithm in this case, as the algorithm only flips the leftmost generating chord.

Case 2: v_p is on the right of the root vertex r

Flipping the (v_a, v_b) chord would produce a (v_p, v_q) chord. However, the (v_a, v_p) chord exists and is positioned to the left of where the new (v_p, v_q) chord would be. Since (v_a, v_p) is more to the left than (v_p, v_q) , the flip is invalid according to the algorithm's leftmost-chord-first policy.

Case 3: v_p is at the root vertex r

Flipping the (v_a, v_b) chord would create a (r, v_q) chord, where r is the root vertex. This would give the child triangulation's root vertex better visibility than the parent—the root would be able to see v_q through a chord, which contradicts the tree construction where chords from the root are only added in the root triangulation.

This violates the algorithm's fundamental constraint that all chords emanating from the root vertex are established in the root triangulation and are never created through flipping operations in descendant triangulations.

Conclusion:

In all three cases, flipping a blocking chord in a triangulation tree is impossible according to the algorithm's rules. Therefore, if (v_a, v_b) is a blocking chord (duplicate from previous faces), it will be present in all descendants of T in the local tree, making all descendants invalid.

By pruning such branches, we eliminate no valid triangulations, preserving completeness.

No Duplicates:

The local genealogical tree structure for each face ensures unique parent-child relationships: each triangulation (except the root) has exactly one parent obtained by flipping its generating chord. This defines a tree (not a DAG), preventing cycles and duplicates within a single face's triangulation space.

Across faces, the recursive DFS exploration with backtracking ensures that each combination of face triangulations is explored exactly once. The global chord set S is updated and restored correctly during backtracking, maintaining consistency.

Therefore, the algorithm generates all valid triangulations exactly once. $\square$

6 Complexity Analysis

6.1 Time Complexity

Theorem 3 (Amortized Constant Time per Triangulation). *The algorithm has amortized $O(1)$ time complexity per triangulation.*

Proof. Let the biconnected graph have k faces with $n_1, n_2, \dots, n_k$ vertices respectively. Each face F_i has at least $O(n_i)$ triangulations (proven in Lemma 6).

Building Time:

For face F_i with n_i vertices:

- Finding a safe root: $O(n_i)$ time (Algorithm 1)
- Constructing root triangulation: $O(n_i)$ time
- Total building time: $B_i = O(n_i)$

Total Triangulations:

The total number of complete graph triangulations is at least:

$$T \geq \prod_{i=1}^k (n_i - 3) = \Omega \left(\prod_{i=1}^k n_i \right) \quad (1)$$

Amortized Analysis:

The total building time across all faces and all recursive calls is:

$$B_{\text{total}} = \sum_{i=1}^k B_i \cdot \prod_{j=1}^{i-1} n_j \quad (2)$$

$$= B_1 + B_2 \cdot n_1 + B_3 \cdot n_1 \cdot n_2 + \dots + B_k \cdot n_1 \cdot n_2 \dots n_{k-1} \quad (3)$$

Two-Face Analysis ($k = 2$):

For $k = 2$ faces:

$$B_{\text{total}} = n_1 + n_1 \cdot n_2 \quad (4)$$

$$T = n_1 \cdot n_2 \quad (5)$$

Since $n_2 \geq 3$ (minimum cycle size):

$$n_1 \cdot 1 < n_1 \cdot n_2 \quad (6)$$

$$n_1 + n_1 \cdot n_2 < n_1 \cdot n_2 + n_1 \cdot n_2 \quad (7)$$

$$B_{\text{total}} < 2 \cdot n_1 \cdot n_2 = 2T \quad (8)$$

Three-Face Analysis ($k = 3$):

For $k = 3$ faces with n_1, n_2, n_3 vertices:

$$B_{\text{total}} = n_1 + n_1 \cdot n_2 + n_1 \cdot n_2 \cdot n_3 \quad (9)$$

$$T = n_1 \cdot n_2 \cdot n_3 \quad (10)$$

From the two-face case, we know: $n_1 + n_1 \cdot n_2 < 2 \cdot n_1 \cdot n_2$

Therefore:

$$B_{\text{total}} < 2 \cdot n_1 \cdot n_2 + n_1 \cdot n_2 \cdot n_3 \quad (11)$$

Since $n_3 \geq 3$, we have $2 < n_3$:

$$2 \cdot n_1 \cdot n_2 < n_1 \cdot n_2 \cdot n_3 \quad (12)$$

$$B_{\text{total}} < n_1 \cdot n_2 \cdot n_3 + n_1 \cdot n_2 \cdot n_3 \quad (13)$$

$$= 2T \quad (14)$$

General Case (Induction):

Lemma 4 (Building Time Bound). *For any number of faces $k \geq 2$ where each face has at least 3 vertices:*

$$B_{\text{total}} < 2T \quad (15)$$

Proof by Induction. Base case: $k = 2$ is proven above.

Inductive hypothesis: Assume for $k - 1$ faces:

$$n_1 + n_1 \cdot n_2 + \cdots + n_1 \cdot n_2 \cdots n_{k-1} < 2 \cdot n_1 \cdot n_2 \cdots n_{k-1} \quad (16)$$

Inductive step: For k faces:

$$B_{\text{total}} = n_1 + n_1 \cdot n_2 + \cdots + n_1 \cdot n_2 \cdots n_{k-1} + n_1 \cdot n_2 \cdots n_k \quad (17)$$

$$< 2 \cdot n_1 \cdot n_2 \cdots n_{k-1} + n_1 \cdot n_2 \cdots n_k \quad (\text{by hypothesis}) \quad (18)$$

Since $n_k \geq 3$:

$$2 \cdot n_1 \cdot n_2 \cdots n_{k-1} < n_1 \cdot n_2 \cdots n_{k-1} \cdot n_k \quad (19)$$

$$B_{\text{total}} < n_1 \cdot n_2 \cdots n_k + n_1 \cdot n_2 \cdots n_k \quad (20)$$

$$= 2 \cdot n_1 \cdot n_2 \cdots n_k = 2T \quad (21)$$

□

Including output time (constant per triangulation), the total time is $< 3T$, yielding amortized $O(1)$ time per triangulation. □

Corollary 5 (Amortized Linear Time). *For any biconnected planar graph with k faces, the algorithm takes total time $< 3 \times (\text{number of triangulations})$. Therefore, the algorithm has amortized linear time complexity: $O(1)$ per triangulation, or $O(T)$ overall where T is the total number of triangulations.*

Lemma 6 (Minimum Triangulation Count). *A face with n vertices has at least $n - 3$ distinct triangulations.*

Proof. By Theorem 1, every face has at least two safe root vertices v_a and v_c . The v_a -root triangulation has all chords emanating from v_a , while the v_c -root triangulation has all chords from v_c .

At most one chord (v_a, v_c) can appear in both triangulations. All other $n - 4$ chords differ. Transforming one into the other via edge flips requires at least $n - 4$ flips, each producing a distinct valid triangulation.

Including the root: at least $n - 3$ triangulations exist. $\square$

6.2 Space Complexity

Theorem 7 (Linear Space Complexity). *The algorithm uses $O(n)$ space, where n is the total number of vertices.*

Proof. The algorithm stores:

- Vertex information for all faces: $O(n)$
- Global chord set S : A planar graph has at most $3n - 6$ edges, so $O(n)$ chords
- Recursion stack depth: $O(k)$ where k is the number of faces, and $k \leq n$
- Current triangulation chords: $O(n)$

Total space: $O(n)$. $\square$

7 Implementation and Experimental Results

We implemented the algorithm in Python, with comprehensive testing on various biconnected plane graphs. The implementation is available at <https://github.com/TAR2003/ThesisGraph>.

7.1 Implementation Details

Key implementation aspects include:

- **Face Representation:** Each face is stored as a list of vertices in counterclockwise order
- **Global Chord Set:** Implemented using a hash set for $O(1)$ lookup and insertion
- **Triangulation Representation:** Each triangulation is a set of chords (vertex pairs)
- **Edge Flipping:** Efficiently computed by identifying the generating chord and its adjacent triangles
- **Canonical Form:** Each triangulation is stored in a canonical form (sorted chord list) to facilitate duplicate detection and verification

7.2 DFS Exploration Tree Structure

The algorithm explores triangulations using a depth-first search (DFS) tree where:

- **Level 0:** Root node (no faces processed yet)
- **Level i ($1 \leq i \leq k$):** Nodes represent partial triangulations where faces $F_1, \dots, F_i$ are triangulated
- **Level k :** Leaf nodes represent complete triangulations (all faces processed)

At each level i , the algorithm:

1. Finds a safe root vertex for face F_i
2. Constructs the root triangulation
3. Explores the local genealogical tree of face F_i via DFS
4. For each triangulation of F_i , recursively processes face F_{i+1}
5. Backtracks by removing chords from the global set

This creates a multi-level tree where each path from root to leaf represents one complete triangulation of the entire graph.

7.3 GlobalChordSet Management

The GlobalChordSet S plays a crucial role in preventing multi-edge creation:

- **Initialization:** $S = \emptyset$ before processing any face
- **Addition:** When exploring a triangulation T of face F_i , add all chords of T to S
- **Checking:** Before adding any chord, verify it does not already exist in S
- **Removal:** After processing all descendants (backtracking), remove chords of T from S
- **Restoration:** This ensures correct state for exploring alternative triangulations

The dynamic addition and removal of chords during DFS ensures that S always contains exactly the chords from the current path in the exploration tree, enabling efficient conflict detection.

7.4 Test Cases

We tested the algorithm on various graph structures:

- **Simple polygons:** Triangle, square, pentagon, hexagon, heptagon, octagon, nonagon
- **Biconnected graphs with 2 faces:** Various configurations including bow-tie, double-square patterns
- **Complex biconnected structures:** Graphs with 3+ faces sharing edges
- **Graphs with varying connectivity:** From minimal biconnected to near-triconnected graphs
- **Degenerate cases:** Graphs with many pre-existing chords from the input
- **Stress tests:** Large graphs with dozens of vertices and multiple faces

Example test inputs from our test suite:

- **Hexagon:** Single 6-vertex cycle, expected: 14 triangulations
- **Bow-tie:** Two squares sharing a vertex, 2 faces
- **Double-square:** Two squares sharing an edge, 2 faces
- **Complex wheel:** Hub vertex connected to outer cycle with 5 spokes

7.5 Performance Results

Experimental results confirm the theoretical analysis:

- **Completeness:** All triangulations were generated without duplicates (verified by exhaustive enumeration for small instances and canonical form checking for larger instances)
- **Efficiency:** Average time per triangulation remained constant as graph size increased, confirming $O(1)$ amortized time
- **Scalability:** The algorithm successfully handled graphs with dozens of vertices and multiple faces, generating thousands of triangulations efficiently
- **Memory usage:** Linear space complexity confirmed—memory usage scaled linearly with the number of vertices
- **Safe root finding:** Always found safe root vertices in $O(n)$ time, with most faces having multiple safe root options

7.6 Verification and Validation

To ensure correctness, we implemented several validation mechanisms:

1. **Triangulation Validity:** Each output is verified to be a valid triangulation (all faces are triangles, no crossing chords)
2. **Duplicate Detection:** Hash-based duplicate detection confirms no triangulation is output twice
3. **Completeness Checking:** For small graphs, we compare output count with known theoretical bounds
4. **Multi-edge Detection:** Verify that no multi-edges exist in any output triangulation
5. **Planarity Verification:** Confirm all generated triangulations maintain planarity

All tests passed successfully, confirming the algorithm’s correctness and efficiency.

8 Conclusion and Future Work

We have presented the first algorithm for generating all triangulations of biconnected plane graphs with constant amortized time complexity. Our algorithm extends the elegant tree-of-triangulations framework of Parvez, Rahman, and Nakano [8] from triconnected to biconnected graphs by introducing safe root selection and local genealogical trees.

8.1 Key Achievements

1. **Theoretical Foundation:** We have provided rigorous proofs establishing:
 - Existence of at least two safe root vertices in every face (Theorem 1)
 - Completeness of the algorithm without duplications (Theorem 2)
 - Amortized $O(1)$ time complexity per triangulation (Theorem 3)
 - Linear $O(n)$ space complexity (Theorem 7)
2. **Practical Algorithm:** Efficient implementation suitable for real-world applications with extensive experimental validation
3. **Novel Techniques:** Safe root selection and global chord management strategies that can extend to other enumeration problems involving planar graph decompositions
4. **Broader Applicability:** Extension from triconnected to the more general class of biconnected plane graphs, significantly expanding the scope of graphs that can be processed efficiently

8.2 Theoretical Significance

This work makes several theoretical contributions:

- **Generalization:** We show that constant-time triangulation enumeration is achievable beyond the restrictive triconnected case
- **Multi-edge Resolution:** Our safe root concept provides a general strategy for handling chord conflicts in face-decomposed planar graphs
- **Amortized Analysis:** The detailed amortized analysis demonstrates that building time is dominated by output size even with complex face interactions
- **Structural Insights:** The T_m structure analysis reveals fundamental properties of planar graph faces under chord constraints

8.3 Future Directions

Several promising research directions remain:

1. **General Planar Graphs:** Extend the algorithm to simply connected (but not bi-connected) planar graphs by handling cut vertices. This would require decomposing the graph at cut vertices and combining triangulations across components.
2. **Constrained Triangulations:** Generate triangulations satisfying additional constraints such as:
 - Minimum angle constraints (for quality mesh generation)
 - Delaunay property (empty circle criterion)
 - Weight minimization (for optimal triangulations)
 - Forbidden edge constraints
3. **Parallel Enumeration:** Exploit the tree structure for parallel generation of triangulations. The local genealogical trees for different faces can potentially be explored in parallel, with synchronization on the global chord set.
4. **Applications:** Apply the algorithm to:
 - Mesh generation for finite element analysis
 - VLSI layout and floorplanning optimization
 - Graph visualization and aesthetic drawing
 - Computational origami and folding patterns
 - Computational biology (protein structure analysis)
5. **Unlabeled Triangulations:** Extend to generate unlabeled triangulations modulo graph isomorphism, reducing redundancy for symmetric graphs
6. **Counting Triangulations:** Develop efficient algorithms for *counting* (without enumerating) all triangulations, potentially using dynamic programming on the face structure

7. **Random Sampling:** Design algorithms for uniformly random sampling from the space of all triangulations, useful for probabilistic analysis and Monte Carlo methods
8. **Dynamic Triangulation:** Investigate online algorithms that maintain the set of triangulations under graph modifications (edge/vertex additions/deletions)

8.4 Impact

This work contributes to the theoretical understanding of planar graph triangulations and provides a practical tool for computational geometry applications. By solving the multi-edge problem inherent in biconnected graphs, we have broadened the applicability of constant-time triangulation enumeration beyond the restrictive triconnected case.

The safe root vertex concept and local genealogical tree structure introduced here may find applications in other graph enumeration problems where global constraints must be maintained across local decompositions. The techniques developed for managing the global chord set during recursive exploration are applicable to any decomposition-based enumeration algorithm.

8.5 Open Problems

We conclude by highlighting several open questions:

1. What is the exact number of safe root vertices in a face as a function of its size and the number of constraining chords?
2. Can the safe root finding algorithm be improved to $o(n)$ time through preprocessing or advanced data structures?
3. Is there a closed-form formula for the number of triangulations of a biconnected graph given its face structure?
4. Can the algorithm be adapted to non-planar graphs with a fixed embedding on other surfaces (torus, projective plane)?

Acknowledgments

The author gratefully acknowledges the foundational work of Mohammad Tanvir Parvez, Md. Saidur Rahman, and Shin-ichi Nakano, whose elegant algorithm for triconnected plane graphs inspired this research. Special thanks to the faculty and students at the Department of Computer Science and Engineering, Bangladesh University of Engineering and Technology (BUET), for their valuable feedback and support.

References

- [1] David Avis and Komei Fukuda. Reverse search for enumeration. *Discrete Applied Mathematics*, 65(1-3):21–46, 1996.
- [2] Bernard Chazelle. Triangulating a simple polygon in linear time. *Discrete & Computational Geometry*, 6(1):485–524, 1991.

- [3] Mark de Berg, Otfried Cheong, Marc van Kreveld, and Mark Overmars. *Computational Geometry: Algorithms and Applications*. Springer, 3rd edition, 2008.
- [4] Ferran Hurtado and Marc Noy. Graph of triangulations of a convex polygon and tree of triangulations. *Computational Geometry*, 13(3):179–188, 1999.
- [5] Krzysztof Kozminski and Edwin Kinnen. Rectangular dualization and rectangular dissections. *IEEE Transactions on Circuits and Systems*, 32(8):771–781, 1985.
- [6] Zheng Li and Shin-ichi Nakano. Generating all triangulations of plane graphs. *Theoretical Computer Science*, 270(1-2):771–784, 2002.
- [7] Takao Nishizeki and Md Saidur Rahman. *Planar Graph Drawing*. World Scientific, 2004.
- [8] Mohammad Tanvir Parvez, Md. Saidur Rahman, and Shin-ichi Nakano. Generating all triangulations of plane graphs. *Journal of Graph Algorithms and Applications*, 15(3):457–482, 2011.
- [9] Ronald C Read. Every one a winner or how to avoid isomorphism search when cataloguing combinatorial configurations. *Annals of Discrete Mathematics*, 2:107–120, 1978.